

Improved Differential Fault Analysis on LED with Constraint Equations: Towards Reaching its Limit

Fan Zhang, Yiran Zhang
College of Information Science
and Electronic Engineering,
Institute of Cyber Security Research,
Zhejiang University,
Hangzhou, P.R.China,
fanzhang@zju.edu.cn

Xinjie Zhao, Shize Guo
Department of Information Security,
Institute of North Electronic Equipment,
Beijing, P.R.China,
zhaoxinjieem@163.com

Ziyuan Liang, Samiya Qureshi
College of Information Science
and Electronic Engineering,
Zhejiang University,
Hangzhou, P.R.China,
liangziyuan@zju.edu.cn

Abstract—The block cipher LED is well suited for resource-constrained scenarios. However, it is vulnerable to the recent fault attacks and different results have been achieved even under the same fault model. In this paper, a comprehensive investigation is conducted on the fault analysis on LED. A novel differential fault analysis is proposed, which is based on the so-called constraint equations. The proposed attack can combine constraint equations at different levels, pushing the differential fault analysis on LED towards its limit in terms of the time complexity, the data complexity and the remained key search space. Under random nibble fault model, SINGLE fault injection can reduce the key search space of LED-64 to $2^{7.90}$ within 1.89s, compared to $2^{17.65}$ within 7 minutes in prior finest contributions. As to DFA on LED-128, TWO fault injections can reduce the key search space to $2^{15.82}$ within 247.88s, compared to $2^{21.96}$ within 16 minutes in previous work. To the best of our knowledge, the scheme that we proposed is the most efficient fault attack on LED cryptosystems.

Index Terms—fault analysis, lightweight block cipher, LED, constraint equation, security evaluation.

I. INTRODUCTION

In recent years, the highly resource-constrained devices are widely used in typical applications of Internet of Things (IoT) and Cyber Physical Systems (CPS), such as health management, mobility, access cards, etc., which are designed to be low-cost [1]. Meanwhile, the possibility of reading personally-linked information without consent has raised serious privacy concerns. Consequently, certain types of cryptography should be employed to deal with sensitive applications. If not considering the overheads of resources, standard algorithms like AES are well suited. However, even the smallest implementation of AES may exceed the capacity of most constrained devices [2]. As a result, many lightweight block ciphers have been designed, such as PRESENT [3], LED [4], Piccolo [5], etc., which are dedicated to fit the IoT environment. Among them, the cipher of *Light Encryption Device* (LED) is an AES-like cipher proposed by Guo et al. at CHES 2011 [4]. It is a

64-bit block cipher with two primary versions involving 64-bit and 128-bit keys: LED-64 (with 32 rounds) and LED-128 (with 48 rounds).

Security evaluation of lightweight block ciphers can be considered from two aspects. The algorithms can be evaluated from the aspect of mathematical security such as differential, linear, algebraic cryptanalysis, and their variants. And their implementations can be investigated from the aspect of physical security with side channel attack (SCA) and fault attack (FA) [6]. Fault attacks retrieve secret information in a cryptosystem by injecting computational faults and exploiting the incorrect output. Faults can be generated by giving external impact on a device with voltage variation, glitch, laser, etc. As to lightweight block ciphers, their implementation platforms are usually resource-constrained and the protection is normally ineffectively deployed, which are very vulnerable to FA. The first idea of FA was reported by Boneh et al. and the details of FA on RSA were published in [6]. The most well-studied FA is the *differential fault analysis* (DFA) that was proposed for secret-key cryptosystems by Biham and Shamir in 1997 [7]. Since then, most of the modern cryptosystems have been investigated under DFA.

The first DFA on LED was presented by Jovanovic et al. in [8]. It assumes that one nibble fault is injected to the 30-th round of LED-64 when both the value and index of the faulty nibble are unknown. Then, the 64-bit key search space can be reduced to about $2^{19} \sim 2^{26}$ with one single fault injection. The attacks are simulated on a workstation with 2.1 GHz AMD Opteron 6172 and 48 GB RAM. All the attacks can be finished within one minute on average. Under the same fault model in [8], there are some other results about DFA on LED-64 in [9], [10]. The work in [9] demonstrates that three fault injections can reduce the key search space of LED-64 to 2^{26} . While the work in [10] shows that single fault injection can reduce the key search space of LED-64 to $2^7 \sim 2^{20}$ ($2^{14.02}$ on average). Due to the different utilizations of DFA, different results have been achieved even under the same fault model [8]–[10].

Besides DFA, there is also some work that combined DFA with algebraic techniques, which is called *algebraic fault*

TABLE I: Summary of fault attacks on LED.

Version	Model	Number of Injections	Setup	Time ($\bar{t}$)	$\phi(K)$	Reference
LED-64	R_{30} , Unknown	1	2.1 GHz, 48 GB RAM	60s	$2^{19} \sim 2^{26}$	[8]
LED-64	R_{30} , Unknown	3	1.6 GHz, 2 GB RAM	1s	2^{26}	[9]
LED-64	R_{30} , Unknown	1	3.2 GHz, 2 GB RAM	2.7s	$2^7 \sim 2^{20}$ ($2^{14.02}$)	[10]
LED-64	R_{30} , Unknown	1	3.5 GHz, 50 GB RAM	14.67h	-	[11]
LED-64	R_{30} , Unknown	1	2.8 GHz, 4 GB RAM	3m	$2^6 \sim 2^{17}$ ($2^{12.9}$)	[12]
LED-64	R_{29} , Unknown	48	32GB RAM	60s	1	[13]
LED-64	R_{29} , Unknown	1	3.2 GHz, 2 GB RAM	7m	$2^{17.43} \sim 2^{17.72}$ ($2^{17.65}$)	[10]
LED-64	R_{29} , Known	1	2.70 GHz, 128 GB RAM	2.1h	$2^{4.04}$	[14]
LED-64	R_{29}, Known	1	2.8 GHz, 4 GB RAM	0.48s	$2^{4.08}$	This paper
LED-64	R_{29}, Unknown	1	2.8 GHz, 4 GB RAM	1.89s	$2^{7.90}$	This paper
LED-128	R_{46}, R_{42} , Unknown	$N_1 = 3, N_2 = 3$	1.6 GHz, 2 GB RAM	1s	2^{27}	[9]
LED-128	R_{45}, R_{41} , Unknown	$N_1 = 48, N_2 = 48$	32GB RAM	—	1	[13]
LED-128	R_{45}, R_{41} , Unknown	$N_1 = 1, N_2 = 1$	3.2 GHz, 2 GB RAM	16m	$2^{15} \sim 2^{27.94}$ ($2^{21.96}$)	[10]
LED-128	R_{45}, R_{41} , Unknown	$N_1 = 2, N_2 = 1$	2.1 GHz, 48 GB RAM	—	$2^{26.97}$	[15]
LED-128	R_{45}, R_{41}, Unknown	$N_1 = 1, N_2 = 1$	2.8 GHz, 4 GB RAM	247.88s	$2^{15.82}$	This paper
LED-128	R_{45}, R_{41}, Unknown	$N_1 = 2, N_2 = 1$	2.8 GHz, 4 GB RAM	2.95s	$2^{7.92}$	This paper
LED-128	R_{45}, R_{41}, Unknown	$N_1 = 1, N_2 = 2$	2.8 GHz, 4 GB RAM	269.77s	1	This paper

analysis (AFA) [11], [12]. In AFA, both the cipher and the fault model are converted into equations and the secret key can be recovered by applying a machine solver to solve these equations. Jovanovic proposed the first AFA on LED-64 in [11] and on average 14.67 hours were required to reveal the key on a workstation having 50 GB RAM. Zhao et al. proposed an improved AFA on LED [12]. The attack assumes that one nibble (byte) fault is injected to the 30-th round of LED-64, and the adversary knows the index of the faulty nibble (byte), but not the value. Using single fault injection, the attack can succeed within three minutes on a personal computer for 89% cases. The key space of LED-64 can be reduced to $2^6 \sim 2^{17}$ ($2^{12.9}$ on average) under the nibble-based fault model, and to $2^6 \sim 2^{14}$ ($2^{10.1}$ on average) under the byte-based fault model.

Due to the constrained resource, lightweight ciphers are normally with a much simpler structure and more iterative rounds. It is quite promising and also reasonable to move the injected faults to deeper rounds. This can also defeat certain fault attack countermeasures which only protect the last few rounds. Whereas existing attacks employ faults injected at the 30-th round of LED-64, there is also some recent work on the injections to the 29-th round [10], [13], [14]. In this scenario, the fault propagation paths will overlap, which is very difficult for the manual analysis of DFA. If a random nibble fault is injected into the 29-th round, the work in [13] proposed an impossible DFA on LED-64, and 48 fault injections can recover the secret key. Under the same fault model, Zhao et al. studied the DFA of LED using the technique of Super-Sbox analysis [10]. One single fault injection can reduce the key search space to $2^{17.43} \sim 2^{17.72}$ ($2^{17.65}$ on average). Ueno et al. proposed another type of efficient DFA on LED-64 [14]. If the index of the faulty nibble is known, one single fault injection can reduce the key search space of LED-64 to $2^{4.04}$ within 2.1 hours.

LED-128 applies two different 64-bit keys (K_0 and K_1) where the master key K can be represented as $K = K_0 \parallel K_1$. In DFA on LED-128, the adversaries have to firstly inject some faults in the last four rounds to recover K_0 and then inject more faults in the last five to eight rounds to recover K_1 . The work in [9] demonstrates that six fault injections (three

in the 46-th round and the other three in the 42-nd round) can reduce the key search space of LED-128 to 2^{27} . Jovanovic et al. introduced a multi-stage fault attack on LED-128 [15]. Three fault injections (two in the 46-th round and one in the 42-nd round) can reduce the 128-bit key search space to $2^{26.97}$ on average. Zhao et al. extended Super-Sbox based DFA to LED-128 and two fault injections (one in the 46-th round and one in the 42-nd round) can reduce the search space to $2^{15} \sim 2^{27.94}$ ($2^{21.96}$ on average) within sixteen minutes [10]. The work in [13] also studied impossible DFA by injecting faults into deeper rounds of LED-128 and 96 fault injections (48 in the 45-th round and the other 48 in the 41-st round) can recover the whole secret key.

Table. I summarizes the previous fault attacks on LED. R_i is the i -th round where the fault is injected to LED-64. R_{i1} and R_{i2} are the respective rounds of the two faults injected to LED-128. *Known* and *Unknown* in the second column indicate the different fault location scenarios, respectively, *i.e.*, whether the index of the injected fault is known to the adversary or not. N_1 and N_2 are the number of injections for the two different faults to LED-128. As for the time, h, m, s are the abbreviations for hours, minutes and seconds, respectively. $\phi(K)$ is the remained key search space of K after FA. From Table. I, different results have been compared in previous work under the same fault model.

This paper aims to provide a generic and quantitative way for FA on LED and to propose new attacks which are pushing the limits of DFA on LED in terms of the time complexity, the data complexity and the remained key search space. Inspired from the work in [16], we introduce *constraint equations* (CEs) in DFA on LED. How to generate and analyze these CEs for fault models are studied systematically. The main contributions of our work include:

- 1) We provide a systematic approach of constructing and solving CEs in DFA on LED. Based on this, we can estimate the number of wrong candidates of the round-key with the computation restrictions in analyzing fault propagation of each round.
- 2) We propose the most efficient DFA of LED-64 under nibble-based fault model. The assumption is that the

adversary can inject a single nibble fault into the 29-th round of LED-64, and he knows the exact index of the faulty nibble (byte), but not the value. We can reduce $\phi(K)$ of LED-64 to $2^{4.08}$ within 0.48 seconds, compared to $2^{4.04}$ within 2.1 hours in [14]. If the exact index of the faulty nibble is unknown, $\phi(K)$ of LED-64 can be reduced to $2^{7.90}$ within 1.89s, compared to $2^{17.65}$ within 7 minutes in [10].

- 3) We propose the most efficient DFA of LED-128 under nibble-based fault model. Assume the adversary can inject one nibble fault into the 45-th round of LED-128 and one nibble fault into the 41-st round, while both the value and the exact index of the faulty nibble are unknown. $\phi(K)$ of LED-128 can be reduced to $2^{15.82}$ and the searching time is only 247.88 seconds on average, compared to $2^{21.96}$ within 16 minutes in [10].

The rest of this paper is organized as follows. The LED cipher is briefly described in Section II. Our fault models are mentioned in Section III. Then Section IV introduces how to generate constraint equations and shows how to use them with an illustrative example. Section V and VI describe the improved DFAs on LED-64 and LED-128 respectively, using the constraint equations that have been defined. Section VII concludes the paper.

II. THE LED CIPHER

LED [4] is a lightweight block cipher with the SPN structure and the 64-bit block size. It has two variants with different key lengths: LED-64 and LED-128. In LED-64, the cipher state is conceptually a matrix whose element is a nibble representing a datum in $GF(2^4)$. For a 64-bit plaintext P , the 16 four-bit nibbles $P_0||P_1||\dots||P_{15}$ are arranged as a 4×4 matrix. Likewise, the 64-bit key $K = k_0||k_1||\dots||k_{15}$, can also be viewed as a 4×4 matrix.

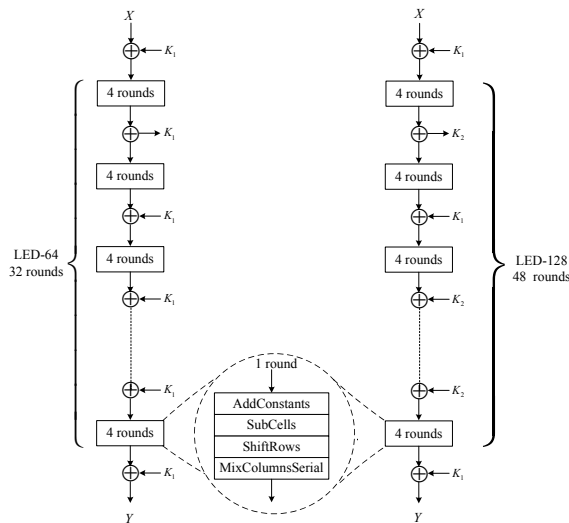


Fig. 1: Overview of LED-64 and LED-128 encryptions [13].

$$P = \begin{pmatrix} P_0 & P_1 & P_2 & P_3 \\ P_4 & P_5 & P_6 & P_7 \\ P_8 & P_9 & P_{10} & P_{11} \\ P_{12} & P_{13} & P_{14} & P_{15} \end{pmatrix}$$

In the encryption of LED, the plaintext P is XORed with K (this operation is denoted as AK) and then goes through the step operation eight times for LED-64 and twelve times for LED-128. Each step consists of four rounds and the state is XOR-ed with K between steps. The value from the last step operation is XORed with K again and becomes the ciphertext.

The encryption procedures of LED-64 and LED-128 are illustrated in Fig. 1. Note that, there is no key scheduling algorithm in LED to generate the round keys. In LED-64, all the round keys K_1 are the same, which are equal to the 64-bit master key K : $K = K_1$. In LED-128, two round keys K_1 and K_2 are alternately used during the encryption, whose concatenation forms the 128-bit master key: $K = K_1||K_2$.

The step operation consists of four round operations, each of which has four inner operations.

- 1) AddConstants (AC). The round constants are XORed with the state.
- 2) SubCells (SC). Each nibble in the state is updated by a lookup using the S-Box of PRESENT [3].
- 3) ShiftRows (SR). Row i of the state is rotated i distance to the left, for $i = 0, 1, 2, 3$.
- 4) MixColumnsSerial (MC). Each column of the state is updated by the multiplication of itself with a fixed matrix M .

$$M = \begin{pmatrix} 4 & 1 & 2 & 2 \\ 8 & 6 & 5 & 6 \\ B & E & A & 9 \\ 2 & 2 & F & B \end{pmatrix}$$

III. OUR FAULT MODEL

For the sake of convenience, we take some notations as follows: Let R_i denote the i -th round, A_i, B_i, C_i, D_i denote the 64-bit output state of AC, SC, SR, MC in R_i , $1 \leq i \leq 32$. $A_i^j, B_i^j, C_i^j, D_i^j$ denote the j -th nibble of A_i, B_i, C_i, D_i , $0 \leq j \leq 15$. $\bar{A}_i^j, \bar{B}_i^j, \bar{C}_i^j, \bar{D}_i^j$ denote the faulty value of $A_i^j, B_i^j, C_i^j, D_i^j$. Let Δ denote the fault difference function. w.l.o.g, suppose one nibble fault is injected to $B_{29}^0 = B_{29}^0 \oplus \bar{B}_{29}^0$. The nonzero value f is the fault difference $\Delta B_{29}^0 = B_{29}^0 \oplus \bar{B}_{29}^0$.

Before describing the fault model, we use the technique of Super-Sbox [10] to give a different view of LED. According to the design of LED, the operations from the R_{31} to the final ciphertext can be expressed as follows:

$$AC \circ SC \circ SR \circ MC \circ AC \circ SC \circ SR \circ MC \circ AK$$

Note that the order of SC and SR in one round can be swapped, thus the above procedure is equivalent to

$$AC \circ SR \circ SSB \circ AK' \circ SR \circ MC$$

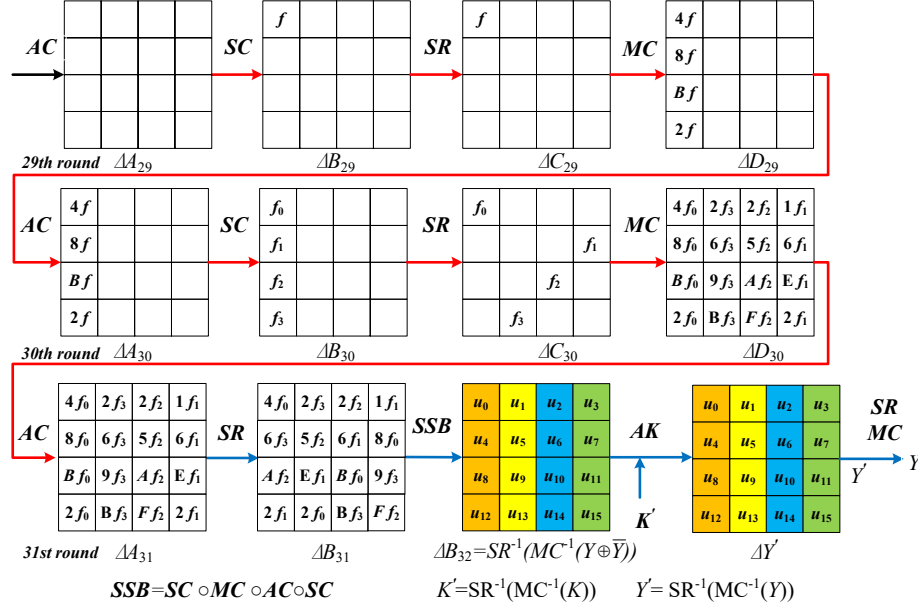


Fig. 2: The procedure of the fault propagation.

where $SSB = SC \circ MC \circ AC \circ SC$. SSB represents a layer of column-wise applications of four 16-bit Super-Sboxes [10], $SSB_0, SSB_1, SSB_2, SSB_3$. AK' denotes the bitwise XOR between B_{32} and the equivalent key $K' = SR^{-1}(MC^{-1}(K))$. The result of AK' is the equivalent ciphertext $Y' = SR^{-1}(MC^{-1}(Y))$. Thus $B_{32} = Y' \oplus K'$. The equivalent fault propagation procedure is shown in Fig. 2.

As done in previous work, we mainly consider the nibble-based fault model in this paper. One nibble output of SC in LED is corrupted by fault injection and the fault difference f is random and unknown. For a plaintext P , the adversary can obtain the corresponding correct ciphertext C and the faulty ciphertext $\bar{C}$ under the same key K . The index of the faulty nibble might be known to the adversary. For example, in [17], Fukunaga and Takahashi showed that they could control the location of a corrupted byte of AES. As to LED, even if the adversary does not know which nibble is corrupted, he can conduct 16 independent and equivalent analyses. For the sake of simplicity, we assume that the adversary knows the index of the corrupted nibble. Next, we use a fault injection to B_{29}^0 , the first nibble of B_{29} of LED-64, to illustrate the fault propagation, as shown in Fig. 2.

Every cell in the 4×4 square of Fig. 2 shows the fault difference of a nibble in the intermediate state. An empty cell indicates that the nibble at that state is correct. The following can be observed from Fig. 2:

- 1) AC and AK affect neither the values nor the locations of the fault differences.
- 2) SC only changes the values of the fault differences.
- 3) SR only changes the locations of the fault differences.
- 4) MC changes both the values and the locations. One nibble fault induced at B_{29} spreads to a column at the

input of R_{30} and to four columns at the input of R_{31} , where each column is affected by a one-nibble fault.

- 5) Since the 32-th round, almost all the nibbles in the states become faulty and the fault nibbles are overlapped.

Among the four operations, SC is the only operation that the fault differences at the output are associated with both the value and the fault differences at the input. This property can be used to recover the input state and then obtain the key.

IV. CONSTRAINT EQUATIONS IN DFA ON LED

A. Core Idea of Constraint Equations

As to DFA on block cipher with S-Boxes, the main difficulty is guessing the input fault differences of S-Box from the correct and faulty ciphertexts. This can be solved by building *constraint equations* (CEs) to eliminate wrong candidates of the input fault differences of S-Box in the last round and then to recover the last roundkey. These CEs can be constructed using the correct ciphertext, faulty ciphertext, roundkeys, and the characteristics of faulty values. Then, the attack can be iterated to the penultimate round and much deeper rounds until the search space of the master key is relatively small for exhaustive search.

To better understand CEs, Table. II lists some of the notations. X_i (Y_i) and $\bar{X}_i$ ($\bar{Y}_i$) are the correct and faulty input (output) of one S-Box. ΔX_i and ΔY_i are the corresponding input and output differences, respectively. Note that different input differences $\Delta X_0, \dots, \Delta X_i$ are actually jointly constrained by the so-called constraint relation **CR** which is denoted as $\mathbf{CR}(\Delta X_0, \dots, \Delta X_i)$. Similarly, given the output values, the input differences can be further constrained by the constraint relation denoted as $\mathbf{CR}(\Delta X|Y, \bar{Y})$. As to one

TABLE II: Notations used in building CEs in DFA.

Variable	Notations
X_i	a correct input of S-Box
$\bar{X}_i$	a faulty input of S-Box
$Y_i = Sbox(X_i)$	a correct output of S-Box
$\bar{Y}_i = Sbox(\bar{X}_i)$	a faulty output of S-Box
$\Delta X_i = X_i \oplus \bar{X}_i$	input difference between X_i and $\bar{X}_i$
$\Delta Y_i = Y_i \oplus \bar{Y}_i$	output difference between Y_i and $\bar{Y}_i$
$\mathbf{CR}(\Delta X_0, \dots, \Delta X_i)$	relation among $\Delta X_0, \dots, \Delta X_i$
$\mathbf{CR}(\Delta X Y, \bar{Y})$	relation among $\Delta X, Y, \bar{Y}$
$\#(\Delta X_i \Delta Y_i = \beta)$	number of candidates for ΔX_i when $\Delta Y_i = \beta$
$\#Y_i (\Delta X_i = \alpha, \Delta Y_i = \beta)$	number of candidates for Y_i when $\Delta X_i = \alpha, \Delta Y_i = \beta$
$\#(\Delta X \Delta Y)$	number of candidates for ΔX for all possible ΔY
$\phi(K \Delta Y \rightarrow \Delta X)$	reduced key entropy of K from analyzing ΔY to ΔX

specific output difference β where $\Delta Y_i = \beta$, the number of possible candidates for ΔX_i is denoted as $\#(\Delta X_i|\Delta Y_i = \beta)$. If the input and output difference are specified as α and β , the number of candidates for Y_i can be denoted as $\#Y_i|(\Delta X_i = \alpha, \Delta Y_i = \beta)$. $\#(\Delta X|\Delta Y)$ denotes the number of candidates for ΔX when taking all possible values of ΔY_i into consideration. The reduced key search space of K from analyzing ΔY to ΔX can be denoted as $\phi(K|\Delta Y \rightarrow \Delta X)$.

For a specific non-zero S-Box output difference β where $\beta \neq 0$, the number of possible S-Box input differences is constrained to a small value, which can be used to build up related CE and further to deduce the secret key. To understand this basic idea, Table. III lists the number of possible input differences $\#(\Delta X_i|\Delta Y_i = \beta)$ of S-Boxes used in LED for all candidates of β . The average value of $\#(\Delta X_i|\Delta Y_i = \beta)$ is about 6.4. This means that if β is specified, the search space of ΔX_i is reduced to 6.4 instead of 16. If the input difference is also specified as α , the number of candidates for Y_i is further reduced. And $\#Y_i|(\Delta X_i = \alpha, \Delta Y_i = \beta) = 2$ for 75% cases and 4 for 25% cases. Note that the analysis can be conducted iteratively. Finally Y_i will be XORed with the last round key and processed with other linear operations, resulting in the ciphertexts. Therefore the last roundkey can be deduced.

TABLE III: The biased distribution of the number of input differences when the output difference is specified.

β	0x01	0x02	0x03	0x04	0x05	0x06	0x07	0x08
$\#(\Delta X_i \Delta Y_i = \beta)$	6	7	7	6	4	7	7	7
β	0x09	0x0a	0x0b	0x0c	0x0d	0x0e	0x0f	
$\#(\Delta X_i \Delta Y_i = \beta)$	7	8	6	7	7	6	4	

Suppose there are m pairs of input and output difference. Therefore, as shown in Eq.(1), we can make the constrain equations (CEs) with those input difference $\Delta X = \{\Delta X_{i1}, \dots, \Delta X_{im}\}$, those output differences $\Delta Y = \{\Delta Y_{i1}, \dots, \Delta Y_{im}\}$, those output values $Y, \bar{Y}$, and those constraint relations $\mathbf{CR}(\Delta Y), \mathbf{CR}(\Delta X), \mathbf{CR}(\Delta X|Y, \bar{Y})$, in order to find the possible candidates for the master key K . Due to the design of LED, K can be the one for LED-64, or referred as K_1 and K_2 for LED-128.

$$\mathbf{CE}(K|\Delta Y \rightarrow \Delta X, \mathbf{CR}(\Delta Y), \mathbf{CR}(\Delta X), \mathbf{CR}(\Delta X|Y, \bar{Y})) \quad (1)$$

Note that $\Delta Y, Y, \bar{Y}$ can be computed from the ciphertexts if the analysis is done in the last round. And the constraint relation $\mathbf{CR}(\Delta Y), \mathbf{CR}(\Delta X), \mathbf{CR}(\Delta X, Y, \bar{Y})$ can be determined by the fault model.

B. Illustrative Example

To illustrate how CEs are constructed and used in analysis, we take DFA with one nibble fault injection to B_{29}^0 in the 29-th round (R_{29}) as an example. The CE of FA on the 29-th round of LED can be constructed from four rounds: $R_{29}, R_{30}, R_{31}, R_{32}$.

(1) Building CEs for the 32-th and 31-th round

Due to the introduction of Super-Sbox, CEs of R_{32} and R_{31} can be combined together, which can be denoted as:

$$\mathbf{CE}(K|\Delta B_{32} \rightarrow \Delta B_{31}, \mathbf{CR}(\Delta B_{32}), \mathbf{CR}(\Delta B_{31}), \mathbf{CR}(\Delta B_{31}|B_{32}, \bar{B}_{32})) \quad (2)$$

In Eq.(2), $\mathbf{CR}(\Delta B_{32})$ can be calculated as that in Eq.(3) where both Y and $\bar{Y}$ are the ciphertexts.

$$\begin{aligned} \Delta B_{32} &= SR^{-1}(MC^{-1}(Y \oplus K)) \oplus SR^{-1}(MC^{-1}(\bar{Y} \oplus K)) \\ &= SR^{-1}(MC^{-1}(Y \oplus \bar{Y})) \end{aligned} \quad (3)$$

$\mathbf{CR}(\Delta B_{31})$ can be calculated as

$$\begin{aligned} \Delta B_{31}^0 &= 4f_0, \Delta B_{31}^1 = 2f_3, \Delta B_{31}^2 = 2f_2, \Delta B_{31}^3 = f_1, \\ \Delta B_{31}^4 &= 8f_0, \Delta B_{31}^5 = 6f_3, \Delta B_{31}^6 = 5f_2, \Delta B_{31}^7 = 6f_1, \\ \Delta B_{31}^8 &= Bf_0, \Delta B_{31}^9 = 9f_3, \Delta B_{31}^{10} = Af_2, \Delta B_{31}^{11} = Ef_1, \\ \Delta B_{31}^{12} &= 2f_0, \Delta B_{31}^{13} = Bf_3, \Delta B_{31}^{14} = Ff_2, \Delta B_{31}^{15} = 2f_1, \\ f_0 &\neq 0, f_1 \neq 0, f_2 \neq 0, f_3 \neq 0 \end{aligned} \quad (4)$$

In Eq.(2), $\mathbf{CR}(\Delta B_{31}|B_{32}, \bar{B}_{32})$ can be denoted as

$$\begin{aligned} \Delta B_{31} &= SSB^{-1}(B_{32}) \oplus SSB^{-1}(\bar{B}_{32}) \\ &= SSB^{-1}(SR^{-1}(MC^{-1}(Y \oplus K))) \\ &\quad \oplus SSB^{-1}(SR^{-1}(MC^{-1}(\bar{Y} \oplus K))) \end{aligned} \quad (5)$$

Note that Eq.(3) and Eq.(5) can be directly determined from ciphertexts Y and $\bar{Y}$ to build a constraint for the unknown K . Eq.(4) can be directly determined from the nibble-based fault model.

According to Eq.(4), for one nibble fault injection, $\phi(K)$ of LED-64 can be reduced to

$$\left(\frac{15^4}{15^3}\right)^4 = (15)^4 = 2^{15.63} \quad (6)$$

(2) Building CEs for the 30-th round

Similarly, the CEs for the 30-th round can be denoted as:

$$\mathbf{CE}(K|\Delta B_{30} \rightarrow \Delta A_{30}, \mathbf{CR}(\Delta B_{30}), \mathbf{CR}(\Delta A_{30}), \mathbf{CR}(\Delta A_{30}|B_{30}, \bar{B}_{30})) \quad (7)$$

where $\mathbf{CR}(\Delta B_{30})$ can be denoted as

$$\begin{aligned}
\Delta B_{30}^0 &= f_0, \Delta B_{30}^1 = 0, \Delta B_{30}^2 = 0, \Delta B_{30}^3 = 0, \\
\Delta B_{30}^4 &= f_1, \Delta B_{30}^5 = 0, \Delta B_{30}^6 = 0, \Delta B_{30}^7 = 0, \\
\Delta B_{30}^8 &= f_2, \Delta B_{30}^9 = 0, \Delta B_{30}^{10} = 0, \Delta B_{30}^{11} = 0, \\
\Delta B_{30}^{12} &= f_3, \Delta B_{30}^{13} = 0, \Delta B_{30}^{14} = 0, \Delta B_{30}^{15} = 0, \\
f_0 &\neq 0, f_1 \neq 0, f_2 \neq 0, f_3 \neq 0
\end{aligned} \quad (8)$$

$\mathbf{CR}(\Delta A_{30})$ can be denoted as

$$\begin{aligned}
\Delta A_{30}^0 &= 4f, \Delta A_{30}^1 = 0, \Delta A_{30}^2 = 0, \Delta A_{30}^3 = 0, \\
\Delta A_{30}^4 &= 8f, \Delta A_{30}^5 = 0, \Delta A_{30}^6 = 0, \Delta A_{30}^7 = 0, \\
\Delta A_{30}^8 &= Bf, \Delta A_{30}^9 = 0, \Delta A_{30}^{10} = 0, \Delta A_{30}^{11} = 0, \\
\Delta A_{30}^{12} &= 2f, \Delta A_{30}^{13} = 0, \Delta A_{30}^{14} = 0, \Delta A_{30}^{15} = 0, \\
f &\neq 0
\end{aligned} \quad (9)$$

$\mathbf{CR}(\Delta A_{30}|B_{30}, \bar{B}_{30})$ can be denoted as

$$\Delta A_{30} = SC^{-1}(B_{30}) \oplus SC^{-1}(\bar{B}_{30}) \quad (10)$$

Note that Eq.(8), Eq.(9) and Eq.(10) can be directly determined from fault propagation feature of the nibble-based fault model. Therefore Eq.(7) forms the combined CEs for the 30-th round, which also reduces the impossible candidates for K .

Then, according to Eq.(9), $\phi(K)$ of LED-64 can be further reduced to $2^{3.91}$:

$$\frac{15^4}{15^3} = 15 = 2^{3.91} \quad (11)$$

If the fault locations are unknown, there are 16 possible fault locations, $\phi(K)$ of LED-64 can be reduced to $15 \times 16 = 240 = 2^{7.91}$.

(3) Building CEs for the 29-th round

As to the CEs for the 29-th round, it can be denoted as:

$$\mathbf{CE}(K|\Delta B_{29} \rightarrow \Delta A_{29}, \mathbf{CR}(\Delta B_{29}), \mathbf{CR}(\Delta A_{29}), \mathbf{CR}(\Delta A_{29}|B_{29}, \bar{B}_{29})) \quad (12)$$

Since the fault is injected to the S-Box output of the 29-th round, there is no need to build CE for $\mathbf{CR}(\Delta A_{29}|B_{29}, \bar{B}_{29})$, and $\Delta A_{29} = 0$. Only CE of $\mathbf{CR}(\Delta B_{29})$ needs to be built:

$$\begin{aligned}
\Delta B_{29}^0 &= f \\
\Delta B_{29}^i &= 0, i \neq 0
\end{aligned} \quad (13)$$

In fact, since $\Delta B_{29} = MC^{-1}(SR^{-1}(\Delta A_{30}))$, the CE of Eq.(13) is already included by Eq.(9). Thus, $\phi(K)$ of LED-64 can not be further reduced in analyzing the faults in the 29-th round.

Eventually, the CE of FA on the 29-th round can be constructed from Eq.(2), Eq.(7), Eq.(12), as shown in Eq.(14). The notation $\uplus$ indicates a merge of two constraint equations.

$$\begin{aligned}
&\mathbf{CE}(K|\Delta B_{32} \rightarrow \Delta B_{29}, \mathbf{CR}(\Delta B_{32}), \mathbf{CR}(\Delta B_{29}), \mathbf{CR}(\Delta B_{29}|B_{32}, \bar{B}_{32})) \\
&= \mathbf{CE}(K|\Delta B_{32} \rightarrow \Delta B_{31}, \mathbf{CR}(\Delta B_{32}), \mathbf{CR}(\Delta B_{31}), \mathbf{CR}(\Delta B_{31}, B_{32}, \bar{B}_{32})) \\
&\uplus \mathbf{CE}(K|\Delta B_{30} \rightarrow \Delta A_{30}, \mathbf{CR}(\Delta B_{30}), \mathbf{CR}(\Delta A_{30}), \mathbf{CR}(\Delta A_{30}|B_{30}, \bar{B}_{30})) \\
&\uplus \mathbf{CE}(K|\Delta B_{29} \rightarrow \Delta A_{29}, \mathbf{CR}(\Delta B_{29}), \mathbf{CR}(\Delta A_{29}), \mathbf{CR}(\Delta A_{29}|B_{29}, \bar{B}_{29}))
\end{aligned} \quad (14)$$

The combination of three levels of constraint equations can push the limit of DFA on LED and further reduce the search space of the master key.

V. APPLICATION TO DFA ON LED-64

We simulate the proposed scheme by a C++ program on a laptop with Intel Core I7-2640M, 2.80 GHz, 4G memory and Windows 7 64-bit OS. As did in [10] and [13], the simulation is on the LED software implementation. Using one nibble fault injection, the statistics of 1000 attacks on LED-64 are shown in Fig. 3. The size of the reduced key search space $\phi(K)$ and the searching time t for our program are given.

A. Experimental Results when the Fault Location is Known

If the location of the nibble fault is known, the size of the reduce key search space $\phi(K)$ is in the range $[2, 36]$. As shown in Fig.3(a), on average $\phi(K) = 16.88 = 2^{4.08}$, which is close to the theoretical value $15 = 2^{3.91}$ in Eq.(11). The average searching time t is about 0.48 seconds as shown in Fig.3(b).

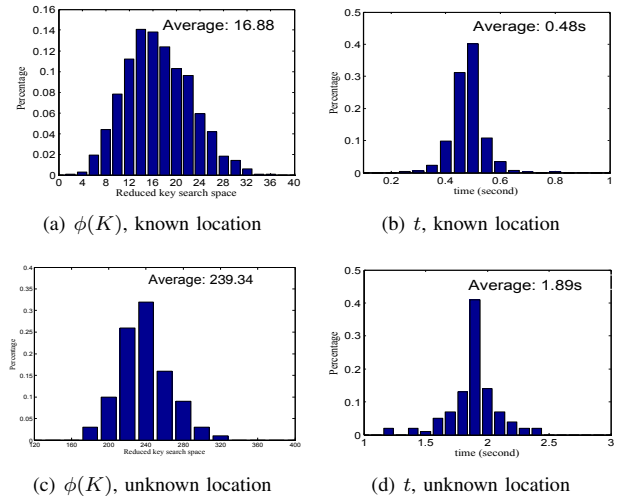


Fig. 3: Results of the attacks on LED-64, $N = 1$.

B. Experimental Results when the Fault Location is Unknown

Under unknown fault location scenario, for one fault injection, there are 16 possible fault nibble locations in B_{29} . The fault propagation path of these locations is shown in Fig. 4 where all possible locations for the fault f are listed in the first state. The subsequent state transitions are described as one row in Fig.4. Note that the operation of Super-Sboxes

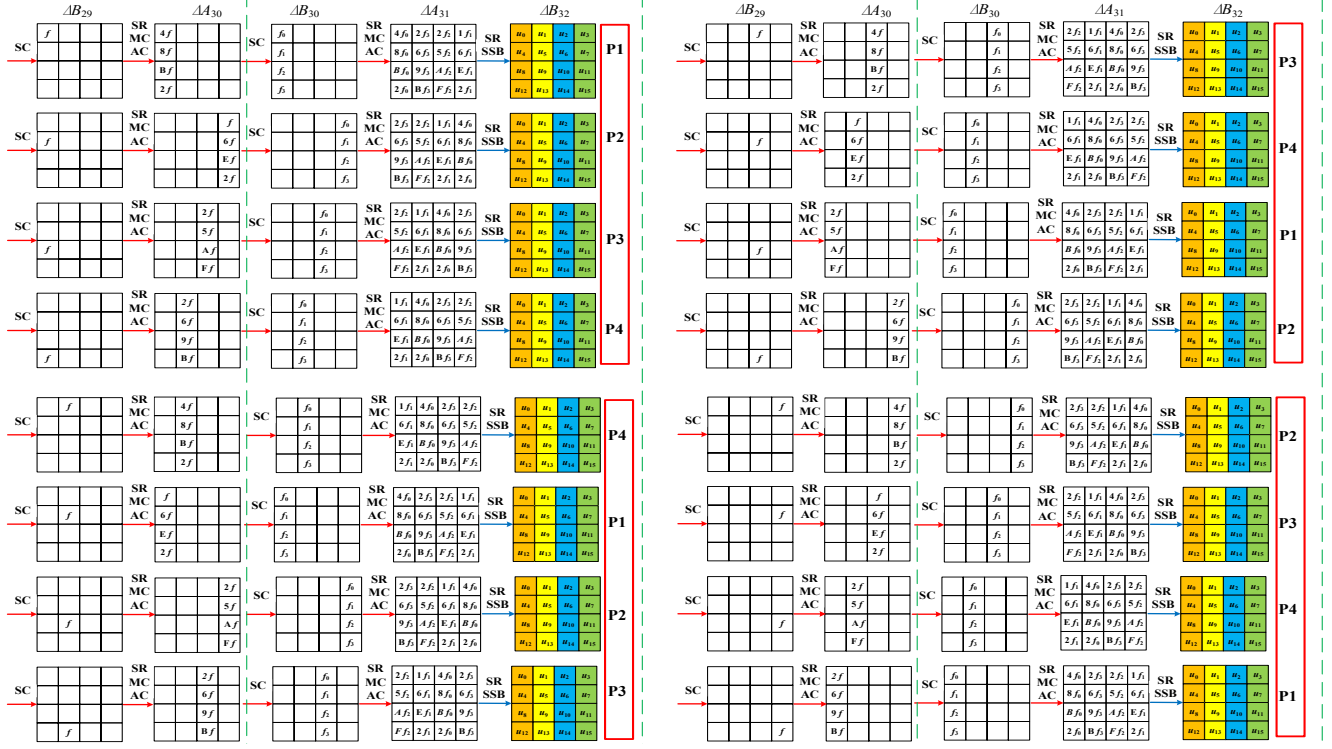


Fig. 4: The fault propagation path of 16 possible fault locations when the specific location is unknown.

is utilized to get ΔB_{32} . We can see that there are only four different patterns (denoted as P1, P2, P3, P4) in ΔA_{31} and sixteen different patterns in ΔA_{30} . The adversary needs to search for these patterns with the CEs in Section IV. In practical experiments, searching for the four different patterns in ΔA_{31} is the most time consuming part. The time complexity of searching for the sixteen different patterns in ΔA_{30} is very small. According to our experience in the fault attacks on LED-64, the analysis time for unknown fault locations is about four times as that for known fault locations.

If the location is unknown, $\phi(K)$ is in the range [160, 360]. As shown in Fig.3(c), the average $239.34 = 2^{7.90}$ is close to the theoretical value $240 = 2^{7.91}$, which is much smaller than $15^{16} = 2^{62.51}$ in [13]. As shown in Fig.3(d), t is about 1.89 seconds on average, which is approximately around four times of that for known fault location scenario. This is consistent to the analysis on fault propagation patterns in Fig.4. TWO faults can extract the whole 64-bit key with 100% success rate and without any brute force. The average searching time is only 3.72 seconds, which is approximately around two times of that in a single fault injection.

VI. APPLICATION TO DFA ON LED-128

As achieved in [10] and [13], the fault attacks on LED-128, the adversary is assumed to be able to inject N_1 and N_2 times of random nibble faults into R_{45} and R_{41} respectively. Let

K_1 and K_2 denote the left and right 64-bit of K in LED-128. Our attacks can be conducted under different number of faults, resulting in different searching time. Suppose the fault location is unknown, the statistics of 1000 attacks are shown in Fig.5. The size of the reduced key search space $\phi(K)$ and the searching time t for our program are given.

A. Results of Attack 1: $N_1 = N_2 = 1$

Since the operations in the last four rounds of LED-128 are exactly the same as those in LED-64, $\phi(K_1)$ can be approximately reduced to $2^{7.90}$. For each candidate of K_1 , the last four rounds of LED-128 can be decrypted and $\phi(K_2)$ can be reduced to $2^{7.90}$ on average. Then, the size of the combinational search space of K_1 and K_2 , i.e., $\phi(K)$ is about $2^{15.82}$ as shown in Fig.5(a). And the searching time is about 247.88 seconds on average as shown in Fig.5(b).

B. Results of Attack 2: $N_1 = 2$ and $N_2 = 1$

Since $N_1 = 2$, $\phi(K_1)$ can be directly recovered. That means $\phi(K_1) = 1$. Then the last four rounds of LED-128 is decrypted and $\phi(K_2)$ can be reduced to $242.12 = 2^{7.92}$ on average as shown in Fig.5(c). Finally, $\phi(K) = 2^{7.92}$. The average searching time is $\bar{t} = 2.95$ seconds and the statistical distribution of all searching times are plotted in Fig.5(d).

C. Results of Attack 3: $N_1 = 1$ and $N_2 = 2$

Similar as Attack 1 where $N_1 = 1$, $\phi(K_1)$ can be reduced to $2^{7.90}$. For each candidate of K_1 , the last four rounds of LED-

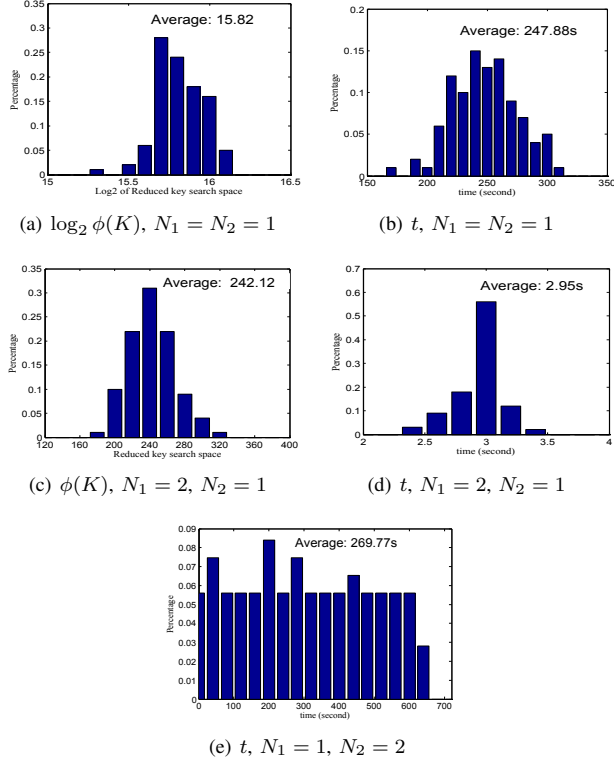


Fig. 5: Results of the attacks on LED-128 (unknown fault location).

128 can be decrypted and $\phi(K_2)$ is computed. For a wrong candidate of K_1 , $\phi(K_2)$ is always equal to zero. Only for a correct candidate of K_1 , the very and single correct candidate of K_2 can be deduced. Therefore, $\phi(K) = 1$ and the success rate is 100%. To further accelerate the searching process, the enumeration of those candidates for K_1 is immediately skipped once the single and correct value of K_2 is achieved. As shown in Fig.5(e), the analysis time is in the range [1, 640] which follows the uniform distribution. The average searching time for LED-128 is about 269.77 seconds with only three injections in total.

VII. CONCLUSION

We have put our best efforts in this paper to provide a generic and quantitative way for conducting fault analysis on LED. The methodology proposes new attacks, pushing the limits of DFA on LED in terms of the time and data complexity, and the remained key search space. The notion of *constraint equations* (CEs) is introduced in DFA on LED and a systematic approach of constructing CEs is provided. As to DFA on LED-64, assuming the adversary can inject a random nibble fault into the 29-th round, if the location of the faulty nibble is unknown, $\phi(K)$ can be reduced to $2^{7.90}$ within 1.89s and single fault injection, compared to $2^{17.65}$ within 7 minutes in [10]. As to DFA on LED-128, $\phi(K)$ can be reduced to $2^{15.82}$ within 247.88s and two fault injections, compared to

$2^{21.96}$ within 16 minutes in [10]. As per survey conducted, these are the most efficient DFAs on LED.

REFERENCES

- [1] S. Bhasin, J.-L. Danger, S. Guilley, and Z. Najm, "A low-entropy first-degree secure provable masking scheme for resource-constrained devices," in *Proceedings of the Workshop on Embedded Systems Security*. ACM, 2013, p. 7.
- [2] A. Moradi, A. Poschmann, S. Ling, C. Paar, and H. Wang, "Pushing the limits: a very compact and a threshold implementation of aes," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2011, pp. 69–88.
- [3] A. Bogdanov, L. R. Knudsen, G. Leander, C. Paar, A. Poschmann, M. J. Robshaw, Y. Seurin, and C. Vikkelsoe, "Present: An ultra-lightweight block cipher," in *International Workshop on Cryptographic Hardware and Embedded Systems*. Springer, 2007, pp. 450–466.
- [4] J. Guo, T. Peyrin, A. Poschmann, and M. Robshaw, "The led block cipher," in *Cryptographic Hardware and Embedded Systems – CHES 2011*, B. Preneel and T. Takagi, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 326–341.
- [5] K. Shibutani, T. Isobe, H. Hiwatari, A. Mitsuda, T. Akishita, and T. Shirai, "Piccolo: an ultra-lightweight blockcipher," in *International Workshop on Cryptographic Hardware and Embedded Systems*. Springer, 2011, pp. 342–357.
- [6] D. Boneh, R. A. DeMillo, and R. J. Lipton, "On the importance of checking cryptographic protocols for faults," in *International conference on the theory and applications of cryptographic techniques*. Springer, 1997, pp. 37–51.
- [7] E. Biham and A. Shamir, "Differential fault analysis of secret key cryptosystems," in *Annual international cryptology conference*. Springer, 1997, pp. 513–525.
- [8] P. Jovanovic, M. Kreuzer, and I. Polian, "A fault attack on the led block cipher," in *International Workshop on Constructive Side-Channel Analysis and Secure Design*. Springer, 2012, pp. 120–134.
- [9] W. Li, D.-W. Gu, C. Zhao, Z.-Q. Liu, and Y. Liu, "Security analysis of the led lightweight cipher in the internet of things," *Jisuanji Xuebao (Chinese Journal of Computers)*, vol. 35, no. 3, pp. 434–445, 2012.
- [10] G. Zhao, R. Li, L. Cheng, C. Li, and B. Sun, "Differential fault analysis on led using super-sbox," *IET Information Security*, vol. 9, no. 4, pp. 209–218, 2014.
- [11] P. Jovanovic, M. Kreuzer, and I. Polian, "An algebraic fault attack on the led block cipher," *IACR Cryptology ePrint Archive*, vol. 2012, p. 400, 2012.
- [12] X. Zhao, S. Guo, F. Zhang, Z. Shi, C. Ma, and T. Wang, "Improving and evaluating differential fault analysis on led with algebraic techniques," in *Fault Diagnosis and Tolerance in Cryptography (FDTC), 2013 Workshop on*. IEEE, 2013, pp. 41–51.
- [13] W. Li, W. Zhang, D. Gu, Y. Cao, Z. Tao, Z. Zhou, Y. Liu, and Z. Liu, "Impossible differential fault analysis on the led lightweight cryptosystem in the vehicular ad-hoc networks," *IEEE Transactions on Dependable and Secure Computing*, vol. 13, no. 1, pp. 84–92, 2016.
- [14] R. Ueno, N. Homma, and T. Aoki, "Efficient dfa on spn-based block ciphers and its application to the led block cipher," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. 98, no. 1, pp. 182–191, 2015.
- [15] P. Jovanovic, M. Kreuzer, and I. Polian, "Multi-stage fault attacks on block ciphers," *IACR Cryptology ePrint Archive*, vol. 2013, p. 778, 2013.
- [16] C. H. Kim, "Differential fault analysis of aes: Toward reducing number of faults," *Information Sciences*, vol. 199, pp. 43–57, 2012.
- [17] T. Fukunaga and J. Takahashi, "Practical fault attack on a cryptographic lsi with iso/iec 18033-3 block ciphers," in *Fault Diagnosis and Tolerance in Cryptography (FDTC), 2009 Workshop on*. IEEE, 2009, pp. 84–92.